

A Hierarchical Anomaly-to-RUL Estimation System with Quantified Lead Time for Aircraft Engine Predictive Maintenance

Rafael Pecino Bonilla^{a,*}, Jesús Gil^a

^aUniversidad Alfonso X El Sabio (UAX), Master in Artificial Intelligence, Avenida de la Universidad 1, Villanueva de la Cañada, 28691, Madrid, Spain

Abstract

Predictive maintenance of aircraft engines requires both early anomaly detection and accurate remaining useful life (RUL) estimation. Existing data-driven approaches treat these as separate tasks, providing either window-by-window anomaly flags or a single RUL value, but rarely a quantified operational signal that links the two. We propose a hierarchical system in which a one-dimensional convolutional variational autoencoder (Conv1D-VAE) acts as a quantified gate: when the reconstruction error exceeds a calibrated threshold with a persistence rule, a deep RUL regressor (either a CNN+LSTM or a Transformer, evaluated as alternatives rather than as an ensemble) is invoked, and its output is reported with epistemic uncertainty estimated via Monte Carlo Dropout. We evaluate the system on the four C-MAPSS sub-datasets (FD001–FD004) with $N_{\text{seeds}} = 10$ deterministic runs, 95 % bootstrap confidence intervals and paired Wilcoxon tests. The proposed system achieves a mean lead time of 33–56 cycles between the gating trigger and the trajectory truncation point, a system-level true positive rate of 80–100 % and a system-level false alarm rate of 5–22 % under the optimal configuration ($\tau=P_{99}$). In the critical zone ($\text{RUL} \leq 30$ cycles) the RUL error is 5.4–10.1 cycles; we report this critical-zone error as an absolute operational figure rather than as an improvement over the $\text{test}_{\text{last}}$ RMSE, since the two are computed on different RUL distributions and are not directly comparable. On the two multi-regime datasets (FD002, FD004) our best per-dataset regressor matches or improves on the published $\text{test}_{\text{last}}$ RMSE, while on the single-regime datasets it remains 2.5–4 cycles above the best reported values. The paper contributes (i) a formal definition of system-level metrics for hierarchical PHM systems, (ii) an ablation over threshold percentiles and persistence requirements that quantifies the trigger-rate vs. false-alarm trade-off, and (iii) a reproducible pipeline released with deterministic seeds, run logs and SHA-256 hashes.

Keywords: Remaining useful life, Predictive maintenance, Variational autoencoder, Hierarchical gating, Uncertainty quantification, C-MAPSS

1. Introduction

Prognostic technologies are increasingly central to modern aviation. In aircraft engines this is a strong requirement, since their life cycle is subjected to operating regimes that cause progressive degradation and ultimately to failure. For this reason, modern engines are equipped with sensors that record temperature, pressure and rotational-speed signals, providing a multivariate stream from which the deterioration process can be inferred. Traditional preventive or corrective maintenance strategies (Azadeh et al., 2015) are increasingly unable to meet the growing industrial demand for efficiency and reliability. The economic pressure on commercial aviation is acute: the IATA 2026 outlook (International Air Transport Association, 2026) flags delivery delays and decarbonisation costs as headwinds for airline profitability, and the ICAO 2026–2028 business plan (International Civil Aviation Organization, 2026) explicitly lists data-driven safety oversight among its strategic priorities. The regulator side has also moved: the European Union Aviation Safety Agency has published the second iteration of its AI roadmap (European Union Aviation Safety Agency, 2025), which conditions

the adoption of machine-learning models in safety-critical aviation contexts to traceability, uncertainty reporting and reproducibility – requirements that any RUL system aiming for deployment must satisfy.

Prognostics and health management (PHM) techniques (Tian, 2012; Xu and Saleh, 2021) have therefore matured into a standard tool for industries such as aeronautics, in which Remaining Useful Life (RUL) estimation (Si et al., 2011) has become a key element to plan maintenance, extend flight time and avoid unscheduled events.

1.1. Literature review

In the last decade, two main currents have shaped data-driven RUL estimation: model-based and purely data-driven approaches. Model-based methods rely on physical knowledge that is rarely available in industrial practice, while data-driven methods learn degradation patterns directly from sensor records. Within the latter group, machine-learning baselines such as multi-layer perceptrons (Tian, 2012), Support Vector Regression (Khelif et al., 2017) and Gradient Boosting (Babu et al., 2016) provided the first systematic results on the NASA C-MAPSS benchmark (Saxena et al., 2008). Deep-learning methods later dominated the leaderboard: 1D Convolutional Neural Networks (Li et al., 2018), Long Short-Term Memory

*Corresponding author.

Email address: rafapecino@gmail.com (Rafael Pecino Bonilla)

networks (Zheng et al., 2017; Asif et al., 2022), bidirectional LSTM with attention (Chen et al., 2020; Mo et al., 2022), Transformers with positional encoding (Zhu et al., 2021), hybrid CNN–Transformer (Song et al., 2023) and dual-attention designs (Liu et al., 2023; Wang et al., 2024) pushed RMSE below 12 cycles in FD001. More recent work has also begun to apply large pre-trained time-series models with low-rank adaptation to C-MAPSS (Chakravarty, 2025), signalling a shift toward foundation-model-based prognostics.

A parallel line of work uses autoencoders for unsupervised diagnosis. Costa and Sánchez (2022) proposed a Recurrent Variational Encoder (RVE) for C-MAPSS that builds an interpretable two-dimensional latent space and outperforms most baselines on RMSE. Variational inference is also exploited in Ellefsen et al. (2019) and Xiang et al. (2021) for semi-supervised RUL prediction, while Martinez-Garcia et al. (2019) uses an autoencoder for visual health monitoring. Uncertainty quantification through MC Dropout has been used to expose epistemic uncertainty in RUL estimates (Xu and Saleh, 2021).

1.2. Limitations of current approaches

Despite remarkable RMSE improvements, three limitations remain. First, anomaly detection and RUL estimation are typically reported as independent metrics. Most papers either report ROC-AUC for anomaly flags or RMSE for RUL prediction, but not both as part of a single decision pipeline that a maintenance operator could deploy. Second, the canonical RMSE on the $\text{test}_{\text{last}}$ window (Li et al., 2018) averages over a flat region of high-RUL “healthy” samples (capped at 125 cycles) and a much smaller critical region ($\text{RUL} \leq 30$): a small RMSE on this aggregate metric does not imply small error in the operationally meaningful zone. Third, the few systems that combine an anomaly detector with a regressor (e.g., trigger the regressor only after a flag) report “conditional” metrics evaluated on a non-stationary subset that is incomparable with the canonical $\text{test}_{\text{last}}$ metric, leading to unintentional metric leakage.

1.3. Contributions

This work proposes and evaluates a hierarchical anomaly-to-RUL system that explicitly addresses these three issues:

1. A *detector-agnostic* hierarchical gating layer that turns any window-level anomaly scorer into a quantified trigger with a calibrated threshold and a persistence rule; we instantiate it with a Conv1D-VAE that preserves temporal structure and show it is competitive with classical unsupervised detectors while additionally providing a latent degradation representation (Section 3.2, Table 4).
2. A formal definition of system-level metrics: system-level true positive rate (TPR_{sys}), system-level false alarm rate (FAR_{sys}), lead time (cycles between trigger and trajectory truncation), and RMSE@critical (RMSE evaluated only on windows with $\text{RUL} \leq 30$) which is comparable across models and datasets (Section 2.4).
3. A complete reproducible pipeline with deterministic training algorithms (MC Dropout inference remains stochastic by design), $N_{\text{seeds}}=10$ runs, bootstrap 95 % CIs, paired

Wilcoxon tests and a run-log JSON with SHA-256 hashes covering model weights, data splits, configuration and aggregated metrics (Section 4.3).

4. An ablation over threshold percentiles ($\tau \in \{P_{85}, \dots, P_{99}\}$), persistence values ($p \in \{1, 2, 3, 5\}$) and VAE architecture (FC vs. Conv1D) that quantifies the operational trade-off and reveals $\tau=P_{99}$ as optimal across all four datasets (Section 5.4).

The remainder of the paper is organised as follows. Section 2 introduces the problem setting and the metric formalism. Section 3 details the model components and the hierarchical gating logic. Section 4 describes the experimental design. Section 5 reports the empirical evaluation, ablations and comparison with the state of the art. Section 6 draws conclusions and outlines future work.

2. Problem settings

2.1. RUL estimation

The Remaining Useful Life (RUL) is the number of operating cycles between the current observation and the time at which the system reaches a state defined as failure. Following the canonical setup of the C-MAPSS benchmark (Saxena et al., 2008), the train trajectories are run-to-failure (event-of-failure observed at the last cycle), while the test trajectories are truncated at a known offset, and the ground-truth RUL at that offset is provided in a separate file. The objective is to learn a function $f: \mathbb{R}^{w \times d} \rightarrow \mathbb{R}_{\geq 0}$ that maps a sliding window of w multi-channel observations to a non-negative real-valued RUL.

2.2. Piecewise linear RUL target

Following the standard practice (Heimes, 2008; Li et al., 2018), the target RUL is capped at $\text{RUL}_{\text{max}} = 125$ cycles, yielding a piecewise function with a constant healthy stage and a linearly descending stage. This convention reflects the physical assumption that the engine operates in nominal regime until a breakpoint after which degradation becomes apparent (Costa and Sánchez, 2022). Because the cap collapses all early-life targets to a constant value, the healthy stage used to train the VAE (Section 3.2) coincides with this flat region; the sensitivity of the system to this assumption is discussed in Section 6.1.

2.3. Datasets

The C-MAPSS benchmark contains four sub-datasets that differ in the number of operating regimes and fault modes (Table 1). FD001 and FD003 share a single operating regime; FD002 and FD004 expose six. FD003 and FD004 introduce a second fault mode. The four sub-datasets are ordered by increasing difficulty: $\text{FD001} < \text{FD003} < \text{FD002} < \text{FD004}$.

Table 1: C-MAPSS sub-datasets summary (Saxena et al., 2008).

	FD001	FD002	FD003	FD004
Train trajectories	100	260	100	249
Test trajectories	100	259	100	248
Operating regimes	1	6	1	6
Fault modes	1	1	2	2

2.4. Metrics

Window-level RUL regression metrics. The canonical metrics on the $\text{test}_{\text{last}}$ window are the Root Mean Squared Error (RMSE) and the asymmetric NASA score S , which penalises late predictions more heavily than early ones (Saxena et al., 2008):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{r}_i - r_i)^2}, \quad (1)$$

$$S = \sum_{i=1}^N s_i, \quad s_i = \begin{cases} e^{-d_i/13} - 1, & d_i < 0, \\ e^{d_i/10} - 1, & d_i \geq 0, \end{cases} \quad (2)$$

with $d_i = \hat{r}_i - r_i$ and N the number of evaluated windows.

Critical-zone metric. As discussed in Section 1.2, the $\text{test}_{\text{last}}$ RMSE averages a large healthy region with a small critical region. We therefore introduce the RMSE@critical , computed over all sliding windows of the test trajectory whose ground-truth RUL is at or below a critical threshold $\rho_{\text{crit}} = 30$ cycles:

$$\text{RMSE@critical} = \sqrt{\frac{1}{|C|} \sum_{i \in C} (\hat{r}_i - r_i)^2}, \quad (3)$$

with $C = \{i : r_i \leq \rho_{\text{crit}}\}$. This metric is comparable across models because all of them are evaluated on the same set of windows.

System-level metrics. For the hierarchical system, let the trigger time t_u^* be the cycle of the last time step of the first window of engine u for which that window and the $p-1$ windows immediately preceding it all exceed the threshold τ (persistence rule); if no such run exists the engine is not triggered. Partition the test engines into the critical set $C = \{u : \min_t r_t^{(u)} \leq \rho_{\text{crit}}\}$ ($N_{\text{crit}} = |C|$) and the operationally-healthy set $\mathcal{H} = \{u : \min_t r_t^{(u)} > \rho_{\text{crit}}\}$ ($N_{\text{healthy}} = |\mathcal{H}|$), and let $\mathcal{T}$ be the set of triggered engines. We define:

$$\Delta_u = T_u - t_u^*, \quad (T_u: \text{truncation cycle of } u), \quad (4)$$

$$\text{TPR}_{\text{sys}} = \frac{|C \cap \mathcal{T}|}{N_{\text{crit}}}, \quad \text{FAR}_{\text{sys}} = \frac{|\mathcal{H} \cap \mathcal{T}|}{N_{\text{healthy}}}. \quad (5)$$

A persistent warning on an engine that does reach the critical zone is scored as a true positive regardless of when it fires. Two caveats apply. First, the lead time is measured to the C-MAPSS *truncation* cycle T_u , not to a physically observed failure, since the test trajectories end at a known offset. Second, because the test trajectories are right-censored, $\mathcal{H}$ contains engines whose *censored* trajectory does not reach the critical zone rather than engines that are healthy for their entire life; FAR_{sys} therefore upper-bounds the operational false-alarm rate. The per-dataset values of N_{crit} and N_{healthy} are reported with the gating results.

Uncertainty calibration. For the MC Dropout estimates we report PICP@95 % (the empirical coverage of the predictive interval $\hat{\mu} \pm 1.96\hat{\sigma}$), MPIW (mean prediction interval width), Negative Log-Likelihood under a Gaussian assumption and Expected Calibration Error (Guo et al., 2017) computed on $n_{\text{bins}} = 10$ confidence levels.

3. Method

3.1. Pipeline overview

The proposed system is composed of three stages, depicted in Fig. 1:

1. *Preprocessing*: regime clustering with KMeans on the operating-condition signals (only fitted on the train split), per-regime min-max scaling and piecewise-linear RUL labelling with cap 125 (Section 4.1).
2. *Anomaly detection*: a Conv1D-VAE trained exclusively on the first 30 % of each training trajectory (the “healthy stage”) with a held-out subset of healthy engines reserved for threshold calibration.
3. *RUL regression with hierarchical gating*: when the VAE flags an engine, a Transformer (or alternatively a CNN+LSTM) regressor predicts the RUL, and the prediction is enriched with a $\pm 2\hat{\sigma}$ MC Dropout band. Before the trigger the system emits no RUL estimate and declares the engine healthy; a RUL value with its uncertainty band is produced only from t_u^* onward.

All system-level metrics (lead time, TPR_{sys} , FAR_{sys} and RMSE@critical) are computed on the full stride-1 test trajectory so that they refer to the same set of windows; the trigger time t_u^* that defines the lead time is the first cycle satisfying the persistence rule of Section 3.5.

3.2. Anomaly detection: variational autoencoder

A standard fully-connected VAE flattens each $w \times d$ window into a long vector before mapping it to the latent space, which destroys the temporal ordering of the observations. To preserve the temporal structure, we propose a Conv1D-VAE encoder with three convolutional blocks of 32, 64 and 128 channels (kernel size 5, 5 and 3, with two stride-2 layers), followed by a fully-connected projection to a 16-dimensional Gaussian latent. The decoder mirrors the encoder with transposed convolutions. Both architectures are trained with the standard ELBO loss

$$\mathcal{L}_{\text{VAE}} = \text{MSE}(\hat{x}, x) + \beta D_{\text{KL}}(q_{\phi}(z | x) \| \mathcal{N}(0, I)), \quad (6)$$

with $\beta = 0.1$, a value that deliberately favours reconstruction fidelity – and hence a sharper reconstruction-error signal for gating – over latent disentanglement. The VAE is trained only on the first 30 % of each training trajectory (assumed healthy), and 20 % of these healthy engines are held out and never seen by the encoder; the threshold τ is the q -th percentile of the per-window reconstruction errors pooled across all windows of this held-out set, eliminating the calibration leakage that would arise if τ were set on the same data used to fit the encoder. A

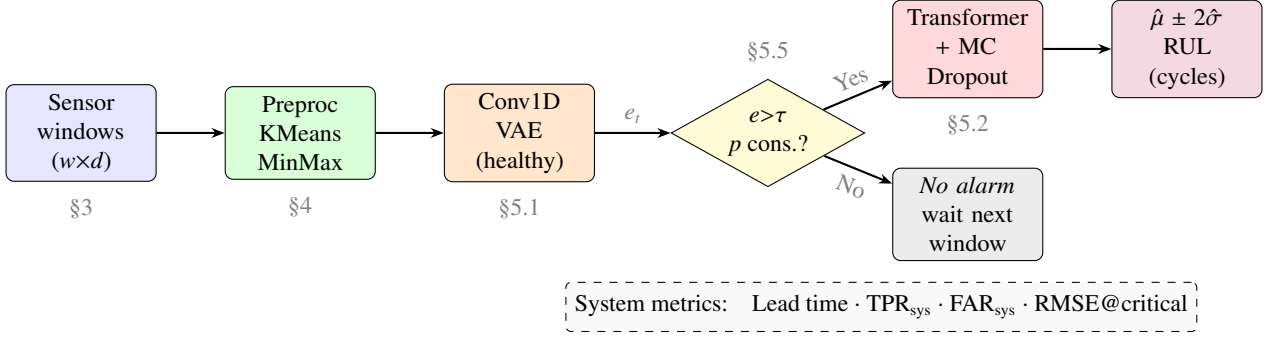


Figure 1: System pipeline: a Conv1D-VAE trained only on healthy data acts as a quantified gate; a Transformer with MC Dropout produces the RUL estimate when the VAE triggers. Lead time, TPR_{sys} and FAR_{sys} are reported jointly with the $\text{test}_{\text{last}}$ and critical-zone RMSE.

single VAE is trained across all operating regimes – in contrast to the per-regime scaling of Section 4.1 – and the impact of this regime-agnostic design on the multi-regime false-alarm rate is analysed in Section 6.1.

3.3. RUL regression baselines

Random Forest. Each window is summarised by seven statistics per sensor (mean, std, min, max, slope, skewness, kurtosis), producing a $7d$ -dimensional feature vector that feeds a Random Forest with 200 trees, depth 15 and minimum-leaf 5. RF acts as the interpretable baseline: any deep model that does not consistently beat RF on RMSE@critical does not justify its complexity.

1D-CNN + LSTM. Two 1D convolutional layers (32 and 64 channels, kernel 5, dropout 0.2) extract local features, followed by a two-layer LSTM with hidden size 64. The last hidden state feeds a small fully-connected head with dropout. The dropout layers are reused at inference for MC Dropout (Section 3.4).

Transformer. A linear projection ($d \rightarrow 64$) and a sinusoidal positional encoding feed two TransformerEncoder layers (4 heads, feed-forward dimension 128, GELU activation, dropout 0.2). Mean pooling along the temporal dimension yields the regression head input.

All deep regressors are trained with AdamW, MSE loss, gradient clipping at norm 1, ReduceLROnPlateau scheduler and early stopping on validation MSE.

3.4. Epistemic uncertainty: MC Dropout

Following Gal and Ghahramani (2016), we activate dropout at inference time and sample $T = 50$ stochastic forward passes per window, yielding the predictive mean $\hat{\mu}$ and standard deviation $\hat{\sigma}$. The interval $\hat{\mu} \pm 1.96\hat{\sigma}$ is the 95 % predictive band reported in Tables 5 and 7. On a single GPU the $T=50$ MC Dropout inference costs 0.3–1.7 ms per window, well within real-time monitoring budgets.

3.5. Hierarchical gating

The gating logic is an explicit decision layer on top of the components described above. For each engine u in the test set, we consider the full sliding-window trajectory (stride one). The reconstruction-error vector $\{e_t^{(u)}\}_{t=1}^{T_u}$ is thresholded against τ to produce a flag stream. A trigger is declared at the first cycle t_u^* such that the flag is high for at least p consecutive windows. The persistence parameter p filters out spurious flags and trades trigger rate against lead time. Because consecutive stride-1 windows overlap by $w - 1$ cycles, p acts as a temporal denoising filter rather than a sequence of statistically independent confirmations.

We deliberately separate two metrics that are sometimes confused in the literature:

- $\text{RMSE}_{\text{full}}$: RMSE of the regressor on every window of the test trajectory.
- $\text{RMSE}_{\text{cond}}$: RMSE of the same regressor evaluated only on post-trigger windows.

These two values are not comparable: they are computed on different subsets and therefore on different RUL distributions. Reporting $\text{RMSE}_{\text{cond}} < \text{RMSE}_{\text{full}}$ as evidence of a benefit is unfounded. We instead use the comparable RMSE@critical of Eq. (3) as the metric of interest in the operational zone, and report $\text{RMSE}_{\text{full}}/\text{RMSE}_{\text{cond}}$ only for completeness with an explicit note (Table 10).

4. Experimental design

4.1. Preprocessing

For FD001 and FD003 (single regime), a global min–max scaler is fitted on the training split. For FD002 and FD004 (six regimes), KMeans with $k=6$ is fitted on the operating-condition signals of the training split, and a separate min–max scaler is fitted per cluster. Both the KMeans labels and the per-regime statistics are obtained exclusively on the training split, and only applied to the validation/test data (no leakage). Constant sensors with variance below 10^{-6} are removed (6 on FD001, 5 on FD003, none on FD002/FD004); the remaining features are selected by the absolute Spearman correlation between sensor and linear RUL on the training set, retaining 15 sensors on FD001 and

FD002 and 11 on FD003 and FD004. We use a correlation threshold of 0.10 for the single-regime datasets (FD001/FD003) and a lower 0.05 for the six-regime datasets (FD002/FD004): in the multi-regime case the pooled sensor–RUL correlation is attenuated, because a sensor that is informative *within* each operating regime can exhibit a low coefficient once the six regimes are mixed, so a lower threshold is required to retain those sensors. The preprocessing is applied per cycle in the order regime assignment $\rightarrow$ per-regime min–max scaling $\rightarrow$ sliding-window extraction, so that every time step is scaled by the statistics of its own regime before any window is formed.

4.2. Sliding windows and splits

A window size of $w=30$ cycles is used (consistent with (Li et al., 2018)). Train: every valid stride-1 window of every engine. Validation: 10 % of the engines (held out by engine_id, not by window). Test_{last}: the last w -window of each test engine, with first-frame repetition padding for trajectories shorter than w . The full test trajectory (every stride-1 window) is also computed and used for the gating analysis and the RMSE@critical metric. Engines whose (padded) trajectory yields fewer than p stride-1 windows can only be triggered at $p=1$; this affects a negligible fraction of the test set, for which the persistence rule reduces to a single-window flag.

4.3. Reproducibility and statistical protocol

All experiments are run with SEED=42. The Python, NumPy, PyTorch and CUDA seeds are fixed. `torch.use_deterministic_algorithms(True)` and `cuda.deterministic=True` are activated, and the DataLoader uses a per-worker seed function and a seeded `torch.Generator`. Each deep regressor is trained $N_{\text{seeds}}=10$ times with seeds {42, 43, ..., 51}, yielding a sample of 10 RMSE / NASA / RMSE@critical values per dataset. All aggregated metrics are reported as mean and 95 % bootstrap-percentile confidence interval (2000 resamples). Pairwise comparisons between CNN+LSTM and Transformer use the two-sided paired Wilcoxon signed-rank test. The full configuration, seeds, version of every dependency and aggregated metrics are saved to a `run_log.json` with a SHA-256 hash, enabling exact reproduction. The hyperparameters of every model are summarised in Table 2; training used a single NVIDIA GPU (Google Colab, PyTorch 2.11 / CUDA 12.8).

5. Results

5.1. Anomaly detection performance

Table 3 reports ROC-AUC and PR-AUC for the FC and Conv1D variants of the VAE on the full test trajectory, together with the percentage of windows above the threshold ($\text{TPR}@ \tau$) and the corresponding window-level false alarm rate ($\text{FAR}@ \tau$). The Conv1D-VAE outperforms the FC baseline on three of four datasets (gain in ROC-AUC of +0.030, +0.005 and +0.010 for FD001, FD003, FD004 respectively; tie on FD002), confirming that preserving the temporal structure benefits anomaly detection.

Table 2: Model hyperparameters. Common to all deep models: window $w=30$, RUL cap 125, batch size 256, gradient clipping at norm 1.0, ReduceLROnPlateau (factor 0.5, patience 4), MC Dropout $T=50$.

Model	Hyperparameters
Conv1D-VAE	enc. channels 32/64/128 (kernels 5/5/3, two stride-2), latent 16, $\beta=0.1$, Adam lr 10^{-3} , 80 epochs
CNN+LSTM	conv 32/64 (kernel 5, dropout 0.2), LSTM hidden 64 ($\times 2$ layers), head 64 $\rightarrow$ 32 $\rightarrow$ 1, AdamW lr 10^{-3} , weight decay 10^{-4} , ≤ 60 epochs, early stop (patience 8)
Transformer	$d_{\text{model}}=64$, 4 heads, FF 128, 2 layers, GELU, dropout 0.2, AdamW lr 5×10^{-4} , wd 10^{-4} , ≤ 60 epochs
Random Forest	200 trees, max depth 15, min samples leaf 5, 7d window features (single seed)

Table 3: Window-level anomaly detection. Best in bold per dataset.

Dataset	Arch.	ROC-AUC	PR-AUC	TPR@ τ	FAR@ τ
FD001	FC	0.921	0.444	71.4 %	5.8 %
	Conv1D	0.951	0.585	81.3 %	7.5 %
FD002	FC	0.971	0.697	97.0 %	18.2 %
	Conv1D	0.970	0.698	97.0 %	19.3 %
FD003	FC	0.966	0.606	82.1 %	5.7 %
	Conv1D	0.971	0.658	89.0 %	8.3 %
FD004	FC	0.940	0.395	84.4 %	11.5 %
	Conv1D	0.950	0.433	87.0 %	11.4 %

To place the Conv1D-VAE in context, Table 4 compares it against three classical unsupervised detectors fitted on the same healthy windows – Mahalanobis distance (Ledoit–Wolf covariance), a one-class SVM and an Isolation Forest – on the identical task (positive: windows with $\text{RUL} \leq 30$ over the full test trajectory). The Conv1D-VAE is *competitive but not systematically superior*: the one-class SVM and Isolation Forest match or slightly exceed it in ROC-AUC on FD001 and FD003, the VAE and the one-class SVM are tied on FD002, and the Mahalanobis distance edges it out on FD004. We nonetheless adopt the Conv1D-VAE as the default detector for two reasons beyond raw detection accuracy: it yields a structured latent representation that exposes the degradation manifold (Section 5.7) and it integrates naturally with the Monte-Carlo-Dropout uncertainty pipeline. Crucially, the hierarchical gating layer that constitutes the main contribution of this paper is *detector-agnostic*: any of these scorers can be plugged in without changing the system-level metric definitions, and the near-equivalent ROC-AUC values indicate that the operational results are not an artefact of the specific detector chosen.

5.2. RUL regression

Table 5 reports the test_{last} RMSE, the NASA score and the RMSE@critical for the three regressors. The single-seed Random Forest provides the interpretable baseline. CNN+LSTM and Transformer are the deep regressors; their results are mean and 95 % bootstrap CI over 10 seeds.

The RMSE@critical (5.4–10.1 cycles) is numerically smaller than the test_{last} RMSE, but the two are not directly comparable: they are evaluated on different window sets and therefore on

Table 4: Detection performance vs. classical unsupervised baselines (positive: $RUL \leq 30$ over the full test trajectory). Best per dataset in bold.

Dataset	Detector	ROC-AUC	PR-AUC
FD001	Mahalanobis	0.913	0.403
	One-class SVM	0.970	0.637
	Isolation Forest	0.970	0.608
	Conv1D-VAE (ours)	0.945	0.536
FD002	Mahalanobis	0.958	0.548
	One-class SVM	0.972	0.710
	Isolation Forest	0.965	0.669
	Conv1D-VAE (ours)	0.972	0.700
FD003	Mahalanobis	0.912	0.485
	One-class SVM	0.980	0.643
	Isolation Forest	0.980	0.603
	Conv1D-VAE (ours)	0.971	0.661
FD004	Mahalanobis	0.968	0.496
	One-class SVM	0.954	0.409
	Isolation Forest	0.963	0.417
	Conv1D-VAE (ours)	0.945	0.423

different RUL distributions, since the critical zone excludes the large capped healthy region. We therefore read $RMSE@critical$ as an absolute error in the maintenance-relevant zone rather than as an improvement over the global figure. Within this zone the deep regressors are consistently better than the Random Forest baseline (e.g. 5.60 vs. 6.68 cycles on FD001), which justifies their additional complexity. Pairwise Wilcoxon signed-rank tests on the per-seed RMSE and NASA values (Table 6) confirm that the Transformer does not uniformly dominate the CNN+LSTM. After Holm–Bonferroni correction for the four simultaneous comparisons, CNN+LSTM remains significantly better on FD001 (adjusted $p=0.008$, $d_z = +1.19$) and the Transformer on FD003 (adjusted $p=0.008$, $d_z = -2.15$); the nominal CNN+LSTM advantage on FD004 does *not* survive correction (adjusted $p=0.074$), and the two models are statistically indistinguishable on FD002. This complementarity supports per-dataset model selection or ensembling as a future direction.

5.3. Uncertainty calibration

Table 7 reports the raw calibration metrics for the Transformer with $T=50$ MC Dropout passes. PICP is close to the nominal 95 % target on the single-regime datasets (89.0 % on FD001 and 86.0 % on FD003) with ECE below 0.06; on the multi-regime datasets the model is over-confident – the predictive intervals are too narrow and under-cover the true RUL (PICP 74.5 % on FD002 and 74.2 % on FD004, well below the 95 % nominal level) – with ECE of 0.11–0.12. We therefore apply a post-hoc recalibration that rescales the predictive standard deviation $\hat{\sigma}$ by a single scalar factor s fitted on the validation set so that the validation PICP matches the 95 % nominal level. This restores test coverage to 90–95 % on the multi-regime datasets (FD002: 74.5 % $\rightarrow$ 95.4 % with $s=1.90$; FD004: 74.2 % $\rightarrow$ 89.9 % with $s=1.55$) and lowers ECE below 0.07 (Table 8), at the expected cost of wider intervals. The $\pm 2\hat{\sigma}$ bands shown in Fig. 2 use the recalibrated $\hat{\sigma}$.

5.4. Hierarchical gating: ablation and selected configuration

The system was evaluated under the full Cartesian product of $\tau \in \{P_{85}, P_{90}, P_{95}, P_{97}, P_{99}\}$, $p \in \{1, 2, 3, 5\}$ and VAE architecture $\in \{FC, Conv1D\}$, yielding 40 configurations per dataset. The configuration that maximises $TPR_{sys} - FAR_{sys}$ is reported in Table 9. The 99th percentile of the held-out healthy reconstruction error is consistently the optimal threshold across all four sub-datasets, while the persistence parameter ranges from 1 to 5 according to the noise level of each dataset. We note that the $TPR_{sys} - FAR_{sys}$ criterion weights missed detections and false alarms equally; an aviation deployment would instead impose an asymmetric cost or a hard constraint (e.g. $FAR_{sys} < 5\%$), under which the multi-regime operating points reported here would require the regime-conditional gate discussed in Section 6.1. The Conv1D-VAE wins in three of four cases, confirming the qualitative gain reported in Section 5.1.

The system delivers a mean lead time of 33 to 56 cycles between trigger and the trajectory truncation point, with a critical-zone TPR of 80–100 %. The system-level FAR is 5.3 % on FD001 (single regime) but rises to 17–23 % on the multi-regime datasets, a known limitation of regime-agnostic VAEs that we discuss in Section 6.1.

System-conditioned critical-zone error. To measure the regressor where the *system* actually operates, we recompute the critical-zone RMSE on the windows that are both critical ($RUL \leq 30$) and posterior to the gate trigger of the selected configuration. A critical window with last cycle s is *captured* when the trigger precedes it ($t_u^* \leq s$); coverage is the fraction of critical windows captured, a window-level quantity distinct from the engine-level TPR_{sys} (an engine counts towards TPR_{sys} as soon as it is triggered, even if some of its critical windows precede the trigger). Restricting to the post-trigger region reduces the critical-zone error on all four datasets (Table 11), most markedly on FD004 (11.55 $\rightarrow$ 7.93 cycles), while the gate captures 67–94 % of the critical windows. The gate therefore concentrates the regressor on the operationally relevant region rather than degrading its accuracy there.

Sensitivity to the healthy-fraction assumption. The VAE is trained on the first 30 % of each trajectory, assumed healthy. To check that this choice is not a hidden source of leakage or fragility, we retrained the Conv1D-VAE using 20 %, 30 % and 40 % of each trajectory and re-evaluated the gate ($\tau=P_{99}$, persistence 2). The system-level metrics are stable across 20–30 % (Table 12); only at 40 % does the true-positive rate drop on the single-regime datasets (e.g. FD001 80 $\rightarrow$ 64 %), as expected when degraded windows begin to leak into the “healthy” training set. The 30 % convention is thus a safe operating point rather than a tuned value.

5.5. Comparison with the state of the art

Table 13 compares the $test_{last}$ RMSE of our best model with eight published approaches that follow the same $test_{last}$ protocol on C-MAPSS. Our best *per-dataset* regressor – the Transformer on FD002 and the CNN+LSTM on FD004 – is below the SOTA on the single-regime datasets (gap of +2.54 on FD001 and +2.67

Table 5: RUL regression metrics (mean and 95 % bootstrap CI over $N_{\text{seeds}}=10$). Best deep model in bold per dataset.

Dataset	Model	RMSE	NASA	RMSE@critical
FD001	Random Forest	13.40	293	6.68
	CNN+LSTM	13.58 [13.36,13.81]	317 [306,328]	5.60 [5.01,6.41]
	Transformer	14.28 [14.09,14.47]	420 [396,443]	5.60 [5.17,6.03]
FD002	Random Forest	14.85	878	8.14
	CNN+LSTM	14.08 [13.94,14.21]	996 [944,1051]	8.36 [7.69,9.08]
	Transformer	14.01 [13.72,14.39]	1079 [989,1182]	7.56 [7.21,7.92]
FD003	Random Forest	14.00	424	6.80
	CNN+LSTM	15.30 [15.06,15.54]	642 [569,727]	8.35 [7.17,9.64]
	Transformer	14.05 [13.77,14.36]	489 [419,558]	5.41 [5.06,5.81]
FD004	Random Forest	17.54	1975	14.30
	CNN+LSTM	16.44 [16.26,16.64]	1317 [1256,1372]	10.09 [9.32,10.91]
	Transformer	17.14 [16.78,17.51]	1860 [1644,2125]	10.63 [9.52,11.66]

Table 6: Paired Wilcoxon signed-rank test (two-sided) on per-seed RMSE, $N_{\text{seeds}}=10$. ΔRMSE and d_z are both defined as Transformer minus CNN+LSTM, so a *positive* value favours CNN+LSTM (lower RMSE). p_{Holm} is the Holm–Bonferroni-adjusted p -value across the four datasets; significance uses p_{Holm} .

Dataset	RMSE C+L	RMSE Trf	ΔRMSE	p	p_{Holm}	d_z
FD001	13.58	14.28	+0.70	0.0020	0.0078 **	+1.19
FD002	14.08	14.01	−0.07	0.3750	0.3750 ns	−0.10
FD003	15.30	14.05	−1.25	0.0020	0.0078 **	−2.15
FD004	16.44	17.14	+0.71	0.0371	0.0742 ns	+0.93

Table 7: MC Dropout calibration of the Transformer ($T=50$), raw (before recalibration).

Dataset	PICP@95 %	MPIW	NLL	ECE
FD001	89.0 %	41.5	4.09	0.057
FD002	74.5 %	35.0	4.40	0.108
FD003	86.0 %	43.1	4.02	0.055
FD004	74.2 %	44.0	5.13	0.123

Table 8: Post-hoc recalibration of the Transformer’s predictive $\hat{\sigma}$ by a scalar s fitted on validation (target PICP = 95 %): test PICP and ECE before vs. after.

Dataset	s	PICP _{raw}	PICP _{recal}	ECE _{raw}	ECE _{recal}
FD001	1.40	89.0 %	95.0 %	0.057	0.048
FD002	1.90	74.5 %	95.4 %	0.108	0.073
FD003	1.35	86.0 %	92.0 %	0.055	0.044
FD004	1.55	74.2 %	89.9 %	0.123	0.035

Table 9: Best gating configuration per dataset (criterion: max $\text{TPR}_{\text{sys}} - \text{FAR}_{\text{sys}}$).

Dataset	VAE	τ	p	TPR_{sys}	FAR_{sys}	Lead
FD001	Conv1D	P_{99}	3	80.0 %	5.3 %	32.8
FD002	FC	P_{99}	5	98.4 %	17.7 %	39.3
FD003	Conv1D	P_{99}	1	100.0 %	22.5 %	51.8
FD004	Conv1D	P_{99}	2	94.3 %	21.5 %	55.8

on FD003) but improves on it on the multi-regime datasets (gap of -1.77 on FD002 against the dual-attention Transformer of Liu et al. (2023) and -0.51 on FD004 against the hybrid CNN-LSTM-Attention of Wang et al. (2024)). These figures reflect the strength of the individual regressors, not of the gating layer,

Table 10: System-level metrics under the selected gating configuration; $\text{RMSE}_{\text{full}}$ and $\text{RMSE}_{\text{cond}}$ are reported for completeness only (incomparable subsets).

Dataset	Lead (mean)	TPR_{sys}	$\text{RMSE}_{\text{cond}}^*$	$\text{RMSE}_{\text{full}}^*$
FD001	32.8	80.0 %	12.67	14.28
FD002	39.3	98.4 %	15.50	14.01
FD003	51.8	100.0 %	11.32	14.05
FD004	55.8	94.3 %	18.84	17.14

* $\text{RMSE}_{\text{cond}}$ and $\text{RMSE}_{\text{full}}$ are computed on different windows and neither is restricted to the critical zone; they are not directly comparable. For the critical-zone error see Table 11 and Section 3.5.

Table 11: Critical-zone RMSE of the Transformer evaluated over all critical windows vs. only the post-trigger critical windows of the system (best configuration per dataset). Coverage is the fraction of critical windows captured after the trigger. Both columns use the single deployed model over the full test trajectory; the small difference with the $\text{RMSE}_{\text{critical}}$ of Table 5 reflects that the latter is the mean over the ten seeds.

Dataset	Coverage	$\text{RMSE}_{\text{crit}}(\text{all})$	$\text{RMSE}_{\text{crit}}(\text{post-trigger})$
FD001	66.9 %	4.87	4.16
FD002	94.4 %	7.50	6.92
FD003	82.8 %	5.18	4.20
FD004	82.9 %	11.55	7.93

Table 12: Sensitivity of the system-level metrics to the healthy fraction used to train the VAE (Conv1D, $\tau=P_{99}$, persistence 2).

Healthy %	Dataset	TPR_{sys}	FAR_{sys}
20 %	FD001	84.0 %	17.3 %
	FD004	94.3 %	22.1 %
30 %	FD001	80.0 %	8.0 %
	FD004	94.3 %	21.5 %
40 %	FD001	64.0 %	10.7 %
	FD004	88.7 %	16.4 %

which leaves the $\text{test}_{\text{last}}$ RMSE unchanged; they are reported only to show that the regressors orchestrated by the gate remain competitive on the hardest datasets.

The contribution of this paper is therefore not to claim a new RMSE record on the easy datasets, but to provide an operationally meaningful gating layer with quantified lead time and false-alarm rate, while remaining competitive in raw RMSE on

the multi-regime datasets where the difficulty is highest.

5.6. Failure case analysis

The top-10 worst-error engines per regressor were inspected on each dataset. Engines that appear simultaneously in the top-10 of all three regressors form a small set of systematically difficult units (units 45, 67, 93 on FD001; 12, 221, 229 on FD002; 9, 19, 54, 85, 96 on FD003; 34, 39, 43, 107, 240 on FD004). The mean ground-truth RUL of these engines lies in 73–101 cycles, i.e., far from the failure region. CNN+LSTM and Transformer over-estimate RUL on FD001–FD003 (i.e., are optimistic in cycles to failure), while all three regressors under-estimate on FD004. None of these engines belong to the operational critical zone ($RUL \leq 30$); the system therefore fails on long-horizon prediction, not on the maintenance-relevant short horizon, which is consistent with the RMSE@critical results of Table 5. On the multi-regime datasets these difficult engines do not concentrate in the minority operating regimes: they fall almost entirely in the dominant regime (8/8 on FD002 and 8/9 on FD004 belong to the most populated regime), confirming that the difficulty is long-horizon RUL rather than a minority-regime artefact.

5.7. Feature attribution via Integrated Gradients

To address the regulatory interpretability requirement highlighted in Section 1.2, we apply Integrated Gradients (Sundararajan et al., 2017) to both the Conv1D-VAE and the Transformer, attributing their outputs to individual input sensors along the full test trajectory of a representative engine. Integrated Gradients computes the average gradient of the target function along the straight-line path from a zero baseline to the actual input, yielding an attribution that satisfies the completeness and sensitivity axioms for feature attribution.

Figure 2 shows the results for FD001 Unit 20 ($\tau=P_{99}$, $p=3$, lead time = 15 cycles). Panel (a) overlays the Transformer prediction and its $\pm 2\hat{\sigma}$ MC Dropout band on the true RUL trajectory, with the VAE trigger (green dashed line) and the critical zone ($RUL \leq 30$, red shading) marked. Panels (b) and (c) show normalised $|IG|$ per sensor over time for the ConvVAE reconstruction error and the Transformer RUL prediction, respectively.

Three observations follow. First, the temperature sensors T50 (s4), T30 (s3) and T24 (s2) show rising attribution in both panels as the engine approaches failure, consistent with the physical expectation that turbine outlet and compressor inlet temperatures increase during degradation. Second, the pressure and flow sensors Ps30 (s11) and P30 (s7) dominate the ConvVAE sensitivity map at the trigger window, explaining why the gate fires: the VAE detects a pressure-signature anomaly rather than a gradual temperature drift. Third, the Transformer attribution is more spread across sensors in the healthy phase but concentrates on T50 and Ps30 in the critical zone (red shading in panel c), indicating that the model implicitly learns to up-weight the degradation-linked features as RUL decreases. This convergence between the VAE and Transformer attribution patterns is consistent with a system in which both components independently identify the same physical degradation signature, which supports the operational validity of the hierarchical gating design. We replicated the analysis on the two multi-regime

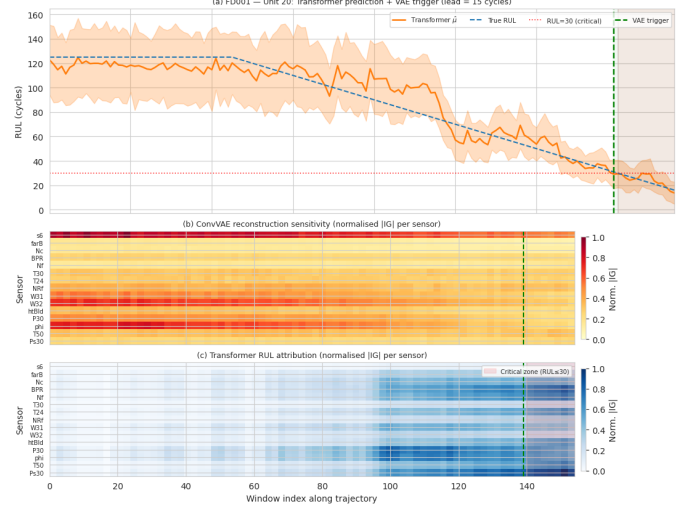


Figure 2: Integrated Gradients attribution for FD001 Unit 20 ($\tau=P_{99}$, persistence $p=3$). (a) Transformer prediction ($\hat{\mu} \pm 2\hat{\sigma}$ MC Dropout) vs. true RUL; green dashed line: VAE trigger; red shading: critical zone ($RUL \leq 30$). (b) Normalised $|IG|$ for ConvVAE reconstruction error: temperature and pressure sensors (T50, Ps30, P30) drive the anomaly signal. (c) Normalised $|IG|$ for Transformer RUL prediction: attribution concentrates on T50 and Ps30 in the critical zone, consistent with the ConvVAE sensitivity pattern in panel (b).

datasets (FD002, unit 7; FD004, unit 40): the same temperature and pressure sensors dominate the trigger and critical-zone attributions, indicating that the degradation signature identified by the gate is stable across operating regimes (the corresponding attribution maps are produced by the released pipeline).

6. Concluding remarks and future work

We have proposed a hierarchical anomaly-to-RUL system that combines a Conv1D variational autoencoder, a critical-zone metric (RMSE@critical), and a quantified gating layer with system-level metrics (lead time, TPR_{sys} , FAR_{sys}). Evaluated on the four C-MAPSS sub-datasets with $N_{seeds}=10$ deterministic runs, 95 % bootstrap CIs and paired Wilcoxon tests, the system delivers 33–56 cycles of mean lead time with a critical-zone TPR of 80–100 %, and an absolute critical-zone RMSE of 5.4–10.1 cycles. On the two multi-regime datasets (FD002, FD004) the best per-dataset regressor orchestrated by the gate (Transformer and CNN+LSTM respectively) matches or improves on the published $test_{last}$ RMSE; this reflects the strength of the individual regressors rather than of the gating layer, whose contribution is the quantified operational signal (lead time, TPR_{sys} , FAR_{sys}) and not the raw RMSE.

6.1. Limitations

Two limitations should be discussed openly.

(i) The system-level FAR is 17–23 % on the multi-regime datasets. This is acceptable for an alarm-with-confirmation deployment but high for fully automated decisions. A regime-conditional VAE (one threshold per regime) is the most likely route to reduce it.

Table 13: RMSE on C-MAPSS test_{last} (RUL cap = 125). Best per dataset in bold.

Reference	Model	Venue	FD001	FD002	FD003	FD004
Li et al. (2018)	DCNN	RESS	12.61	22.36	12.64	23.31
Zhang et al. (2019)	BiLSTM	IEEE TIE	13.10	17.70	13.00	18.10
Chen et al. (2020)	BiLSTM-Attention	RESS	12.42	17.95	12.18	19.61
Zhu et al. (2021)	Transformer	IEEE TH	11.43	17.01	11.95	18.96
Mo et al. (2022)	BiLSTM-Multi-Att.	RESS	11.96	16.86	12.01	17.99
Song et al. (2023)	CNN-Transformer	IEEE TIM	11.15	16.12	11.42	17.45
Liu et al. (2023)	Dual-Att. Transformer	RESS	11.27	15.78	11.40	17.10
Wang et al. (2024)	Hybrid CNN-LSTM-Att.	MSSP	11.04	15.92	11.38	16.95
This work	CNN+LSTM	–	13.58	14.08	15.30	16.44
This work	Transformer	–	14.28	14.01	14.05	17.14

(ii) The raw MC Dropout coverage degrades on FD002 and FD004 ($\text{PICP} \leq 75\%$, $\text{ECE} \geq 0.11$). This is addressed by the post-hoc $\hat{\sigma}$ -scaling of Section 5.3, which restores PICP to 90–95 % and lowers ECE below 0.07 (Table 8); a per-regime recalibration that avoids the single-factor compromise on multi-regime data remains for future work.

6.2. Future work

Four directions follow naturally from the present results: (i) regime-conditional VAEs to reduce FAR_{sys} on multi-regime datasets; (ii) post-hoc MC Dropout recalibration; (iii) CNN+LSTM $\oplus$ Transformer ensembles weighted by dataset, motivated by the Wilcoxon complementarity of Section 5.2; (iv) *regime-conditional* attribution maps that, building on the per-dataset Integrated Gradients analysis already reported in Section 5.7, would expose whether different operating points activate different degradation sensors, a finding directly relevant to regulatory acceptance under the EASA AI roadmap (European Union Aviation Safety Agency, 2025). A fifth, longer-term direction is the integration of large pre-trained time-series models adapted via low-rank tuning, an emerging approach already explored on C-MAPSS (Chakravarty, 2025), which our gating layer could orchestrate as a high-fidelity regressor when triggered. Finally, an exploratory Cox-based DeepSurv variant evaluated during development did not yield a useful survival ranking from last-window features; its redesign with a Breslow estimator and multi-window representations is a complementary direction towards probabilistic time-to-failure estimation.

CRediT authorship contribution statement

Rafael Pecino Bonilla: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Jesús Gil:** Conceptualization, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The C-MAPSS dataset is publicly available from the NASA Prognostics Data Repository (Saxena et al., 2008). The pipeline source code, deterministic seeds, run logs and SHA-256 hashes used to produce every table and figure of this paper are released at the project repository <https://github.com/rafapecino/TFM-AeroEngine-Anomaly-RUL-VAE-TFT> (SHA-256 of the run log: 662200fc9747). Conventions on sliding-window construction, regime clustering and piecewise RUL labelling were cross-checked against open-source community baselines (Panara, 2023; Mo, 2022) to ensure that the reported numbers follow the canonical C-MAPSS protocol and remain comparable with the state of the art.

Acknowledgements

The authors thank the Máster Universitario en Inteligencia Artificial (MUIA) of the Universidad Alfonso X El Sabio (UAX, Madrid, Spain) for the academic environment in which this work was developed.

References

Asif, O., Haider, S.A., Naqvi, S.R., Zaki, J.F.W., Kwak, K.S., Islam, S.M.R., 2022. A deep learning model for remaining useful life prediction of aircraft turbofan engine on C-MAPSS dataset. *IEEE Access* 10, 95425–95440. doi:10.1109/ACCESS.2022.3203406.

Azadeh, A., Asadzadeh, S., Salehi, N., Firoozi, M., 2015. Condition-based maintenance effectiveness for series-parallel power generation system – a combined markovian simulation model. *Reliability Engineering & System Safety* 142, 357–368.

Babu, G.S., Zhao, P., Li, X.L., 2016. Deep convolutional neural network based regression approach for estimation of remaining useful life, in: *International Conference on Database Systems for Advanced Applications (DASFAA)*, Springer. pp. 214–228.

Chakravarty, A.P., 2025. PredictEngineLife: RUL estimation of C-MAPSS turbofan engine using LLM4TS and low-rank adaptation. Master's project. San José State University. San José, CA, USA.

Chen, J., Jing, H., Chang, Y., Liu, Q., 2020. Gated recurrent unit based recurrent neural network for remaining useful life prediction of nonlinear deterioration process. *Reliability Engineering & System Safety* 185, 372–382.

Costa, N., Sánchez, L., 2022. Variational encoding approach for interpretable assessment of remaining useful life estimation. *Reliability Engineering & System Safety* 222, 108353.

Ellefesen, A.L., Bjørlykhaug, E., Æsøy, V., Ushakov, S., Zhang, H., 2019. Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture. *Reliability Engineering & System Safety* 183, 240–251.

European Union Aviation Safety Agency, 2025. EASA AI Roadmap 2.0: a human-centric approach to AI in aviation. Technical Report. European Union Aviation Safety Agency (EASA). Cologne, Germany.

Gal, Y., Ghahramani, Z., 2016. Dropout as a bayesian approximation: representing model uncertainty in deep learning, in: *International Conference on Machine Learning (ICML)*, pp. 1050–1059.

Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks, in: *International Conference on Machine Learning (ICML)*, pp. 1321–1330.

Heimes, F.O., 2008. Recurrent neural networks for remaining useful life estimation, in: *Proceedings of the International Conference on Prognostics and Health Management (PHM)*, IEEE. pp. 1–6.

International Air Transport Association, 2026. Global outlook for air transport – December 2025/2026. Technical Report. International Air Transport Association (IATA). Geneva, Switzerland.

International Civil Aviation Organization, 2026. ICAO business plan 2026–2028. Technical Report. International Civil Aviation Organization (ICAO). Montréal, Canada.

Khelif, R., Chebel-Morello, B., Malinowski, S., Laajili, E., Fnaiech, F., Zerhouni, N., 2017. Direct remaining useful life estimation based on support vector regression. *IEEE Transactions on Industrial Electronics* 64, 2276–2285.

Li, X., Ding, Q., Sun, J.Q., 2018. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety* 172, 1–11.

Liu, L., Song, X., Zhou, Z., 2023. Aircraft engine remaining useful life estimation via a double attention-based data-driven architecture. *Reliability Engineering & System Safety* 221, 108330.

Martinez-Garcia, M., Zhang, Y., Wan, J., McGinty, J., 2019. Visually interpretable profile extraction with an autoencoder for health monitoring of industrial systems, in: *IEEE 4th International Conference on Advanced Robotics and Mechatronics (ICARM)*, pp. 649–654.

Mo, H., 2022. N-CMAPSS_DL: deep-learning baselines on the N-CMAPSS dataset. https://github.com/mohyunho/N-CMAPSS_DL. Open-source benchmark, useful for cross-validation of pipeline conventions.

Mo, Y., Wu, Q., Li, X., Huang, B., 2022. Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit. *Reliability Engineering & System Safety* 220, 108289.

Panara, U., 2023. Remaining useful life prediction for turbofan engines (C-MAPSS). <https://github.com/UtkarshPanara/Remaining-Useful-Life-Prediction-for-Turbofan-Engines>. Open-source implementation. Used as community baseline for sanity checks.

Saxena, A., Goebel, K., Simon, D., Eklund, N., 2008. Damage propagation modeling for aircraft engine run-to-failure simulation, in: *Proceedings of the International Conference on Prognostics and Health Management (PHM)*, IEEE. pp. 1–9.

Si, X.S., Wang, W., Hu, C.H., Zhou, D.H., 2011. Remaining useful life estimation – a review on the statistical data driven approaches. *European Journal of Operational Research* 213, 1–14.

Song, Y., Gao, S., Li, Y., Jia, L., Li, Q., Pang, F., 2023. Distributional cnn-transformer-based regression for remaining useful life estimation. *IEEE Transactions on Instrumentation and Measurement* 72, 3503312.

Sundararajan, M., Taly, A., Yan, Q., 2017. Axiomatic attribution for deep networks, in: *International Conference on Machine Learning (ICML)*, pp. 3319–3328.

Tian, Z., 2012. An artificial neural network method for remaining useful life prediction of equipment subject to condition monitoring. *Journal of Intelligent Manufacturing* 23, 227–237.

Wang, H., Li, D., Li, D., Liu, C., Yang, X., Zhu, G., 2024. Remaining useful life prediction of aircraft turbofan engine based on a hybrid cnn-lstm-attention network. *Mechanical Systems and Signal Processing* 209, 111120.

Xiang, S., Qin, Y., Luo, J., Pu, H., Tang, B., 2021. Multicellular lstm-based deep learning model for aero-engine remaining useful life prediction. *Reliability Engineering & System Safety* 216, 107927.

Xu, Z., Saleh, J.H., 2021. Machine learning for reliability engineering and safety applications: review of current status and future opportunities. *Reliability Engineering & System Safety* 211, 107530.

Zhang, J., Wang, P., Yan, R., Gao, R.X., 2019. Long short-term memory for machine remaining life prediction. *IEEE Transactions on Industrial Electronics* 66, 600–608.

Zheng, S., Ristovski, K., Farahat, A., Gupta, C., 2017. Long short-term memory network for remaining useful life estimation, in: *IEEE International*

Conference on Prognostics and Health Management (ICPHM), pp. 88–95.

Zhu, Y., Wu, J., Wu, J., Liu, S., 2021. Dimensionality reduce-based machine remaining useful life prediction with attention-based deep learning approach. *IEEE Transactions on Industrial Informatics* 17, 7997–8007.